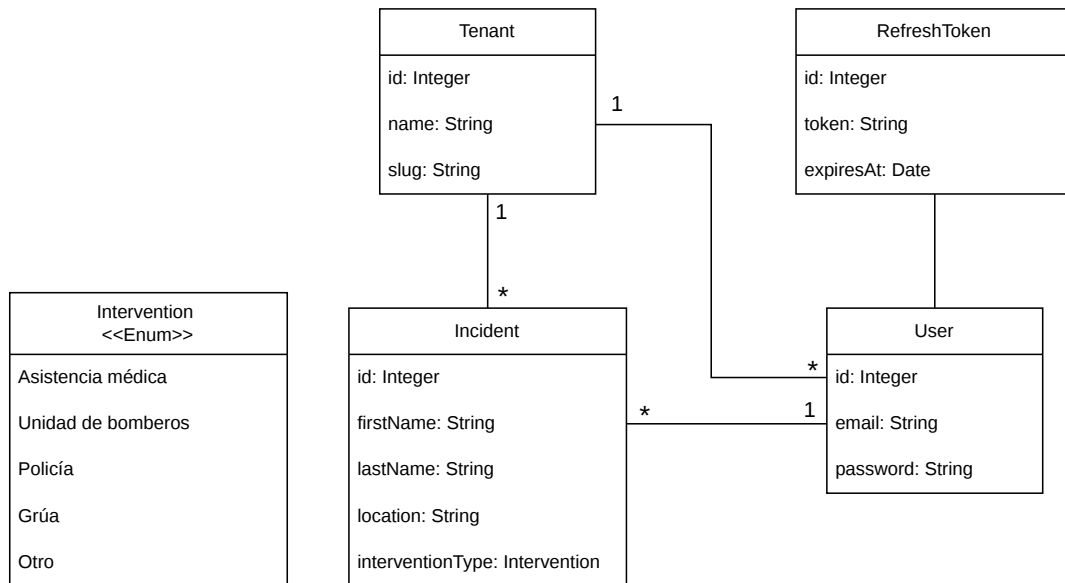


Caso 1 — Gestión de incidencias de tráfico

Backend: `backend/` (puerto 3030) · Frontend: `frontend/` (puerto 3000)

Modelo de datos



Tenant

Representa una organización (empresa, servicio de emergencias, etc.). Es la unidad de aislamiento del sistema multi-tenant.

Campo	Tipo	Restricciones
<code>id</code>	INTEGER	PK, autoincrement
<code>name</code>	STRING	NOT NULL
<code>slug</code>	STRING	NOT NULL, UNIQUE
<code>createdAt</code>	DATE	NOT NULL
<code>updatedAt</code>	DATE	NOT NULL

User

Cada usuario pertenece obligatoriamente a un tenant.

Campo	Tipo	Restricciones
<code>id</code>	INTEGER	PK, autoincrement
<code>email</code>	STRING	NOT NULL, UNIQUE
<code>password</code>	STRING	NOT NULL · almacenado como hash bcrypt (coste 5)
<code>tenantId</code>	INTEGER	FK → Tenants(id), NOT NULL, CASCADE
<code>createdAt</code>	DATE	NOT NULL

Campo	Tipo	Restricciones
updatedAt	DATE	NOT NULL

Incident

Registra un parte de intervención en un accidente de tráfico. Lleva doble FK (`userId` y `tenantId`) para poder filtrar por organización sin hacer JOIN.

Campo	Tipo	Restricciones
id	INTEGER	PK, autoincrement
firstName	STRING	NOT NULL
lastName	STRING	NOT NULL
location	STRING	NOT NULL
interventionType	ENUM	NOT NULL · valores: <code>Asistencia médica</code> , <code>Unidad de bomberos</code> , <code>Policía</code> , <code>Grúa</code> , <code>Otro</code>
userId	INTEGER	FK → Users(id), NOT NULL, CASCADE
tenantId	INTEGER	FK → Tenants(id), NOT NULL, CASCADE
createdAt	DATE	NOT NULL
updatedAt	DATE	NOT NULL

RefreshToken

Almacena los tokens de refresco activos. Se destruye cuando expira o cuando el usuario los rota.

Campo	Tipo	Restricciones
id	INTEGER	PK, autoincrement
token	STRING(512)	NOT NULL, UNIQUE (UUID v4)
userId	INTEGER	FK → Users(id), NOT NULL, CASCADE
expiresAt	DATE	NOT NULL · 7 días desde la creación
createdAt	DATE	NOT NULL
updatedAt	DATE	NOT NULL

Endpoints implementados

Autenticación — `/users`

Método	Ruta	Auth	Descripción
POST	<code>/users/login</code>	—	Valida credenciales y devuelve <code>accessToken</code> (JWT 1 h) + <code>refreshToken</code> (7 días)
POST	<code>/users/register</code>	—	Crea un usuario en un tenant existente (<code>tenantId</code>) o crea el tenant nuevo (<code>tenantName</code>) en una transacción atómica
POST	<code>/users/refresh</code>	—	Rota el access token a partir de un refresh token válido y no expirado
GET	<code>/users/me</code>	Bearer	Devuelve los datos del usuario autenticado junto con el objeto <code>tenant</code>

Body `POST /users/login`

```
{ "email": "usuario@org.com", "password": "secreto" }
```

Body `POST /users/register` (una de las dos opciones de tenant es obligatoria)

```
{ "email": "nuevo@org.com", "password": "secreto", "tenantId": 1 }  
{ "email": "nuevo@org.com", "password": "secreto", "tenantName": "Mi Organización" }
```

Body `POST /users/refresh`

```
{ "refreshToken": "<uuid>" }
```

Tenants — `/tenants`

Método	Ruta	Auth	Descripción
GET	<code>/tenants</code>	—	Lista todos los tenants (id, name, slug) — usado en el dropdown del formulario de registro
GET	<code>/tenants/:id</code>	Bearer	Devuelve un tenant por id
PUT	<code>/tenants/:id</code>	Bearer	Actualiza <code>name</code> y <code>slug</code> de un tenant
GET	<code>/tenants/:id/users</code>	Bearer	Lista los usuarios que pertenecen a un tenant

Incidencias — `/incidents`

Método	Ruta	Auth	Descripción
GET	<code>/incidents</code>	Bearer	Devuelve todas las incidencias del tenant del usuario autenticado, ordenadas por fecha de creación descendente
POST	<code>/incidents</code>	Bearer	Crea una incidencia; <code>tenantId</code> y <code>userId</code> se extraen del JWT, nunca del body
GET	<code>/incidents/:id</code>	Bearer	Devuelve una incidencia verificando que pertenece al tenant del usuario
DELETE	<code>/incidents/:id</code>	Bearer	Elimina una incidencia verificando que pertenece al tenant del usuario

Body `POST /incidents`

```
{  
  "firstName": "Juan",  
  "lastName": "García López",  
  "location": "Av. de la Constitución, 14",  
  "interventionType": "Asistencia médica"  
}
```

Configuración — `/config`

Método	Ruta	Auth	Descripción
GET	<code>/config/intervention-types</code>	—	Devuelve el catálogo de tipos de intervención válidos como array <code>[{ value, label }]</code>

Aislamiento multi-tenant

El aislamiento entre organizaciones se garantiza en tres capas:

1. `tenantId` en el JWT

En el momento del login, el servidor firma un JWT que incluye `{ id, tenantId }`:

```
jwt.sign({ id: user.id, tenantId: user.tenantId }, JWT_SECRET, { expiresIn: '1h' })
```

El cliente no puede alterar este valor sin invalidar la firma del token.

2. Extracción del tenant desde el token, nunca desde el body

El middleware `isLoggedIn` (Passport Bearer) verifica el token en cada petición protegida y adjunta el payload decodificado en `req.user`. Los controladores leen `req.user.tenantId` directamente:

```
// IncidentController – el cliente no puede elegir el tenant
const incident = await Incident.create({
  ...req.body,
  userId: req.user.id,
  tenantId: req.user.tenantId // ← siempre del JWT
})
```

3. Filtrado en base de datos

Todas las consultas de lectura y escritura sobre incidencias incluyen el `tenantId` en la cláusula `WHERE`:

```
// Listado
Incident.findAll({ where: { tenantId: req.user.tenantId } })

// Detalle / borrado – impide que un usuario de otro tenant acceda al recurso
Incident.findOne({ where: { id: req.params.id, tenantId: req.user.tenantId } })
```

Si un usuario autentica su solicitud con un token válido pero intenta acceder a un recurso de otro tenant (manipulando el id en la URL), la consulta no devuelve resultados y el servidor responde `404 Not Found`. No se filtra en capa de aplicación sino directamente en la consulta SQL, lo que elimina la posibilidad de fugas por lógica incorrecta.